Formal verification of systems with unbounded agents

Tephilla Prince

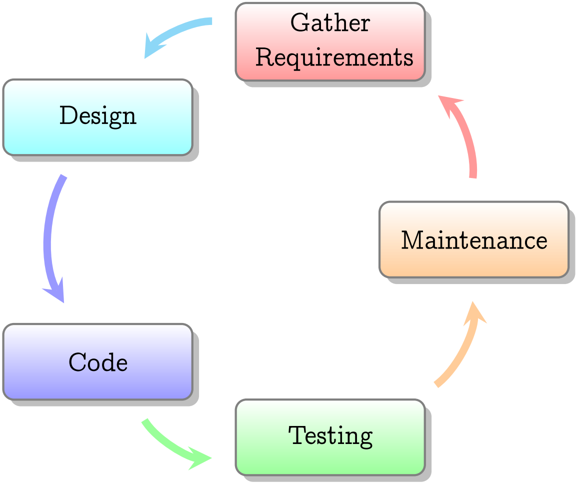
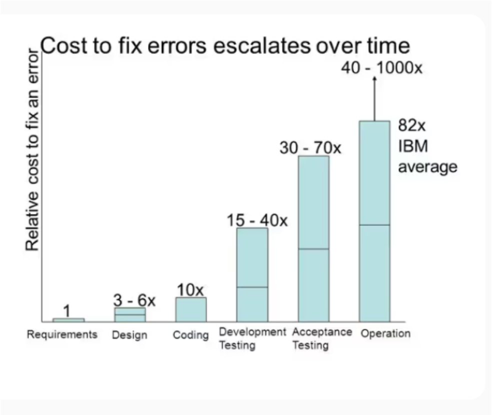


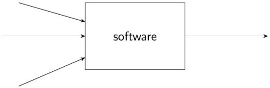
Figure: Traditional Software Development Life Cycle

Error Costs

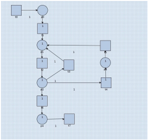
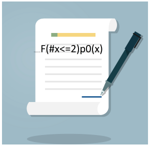


J. Stecklein, “Error cost escalation through the project life cycle, 2010

What is model checking?

Does 

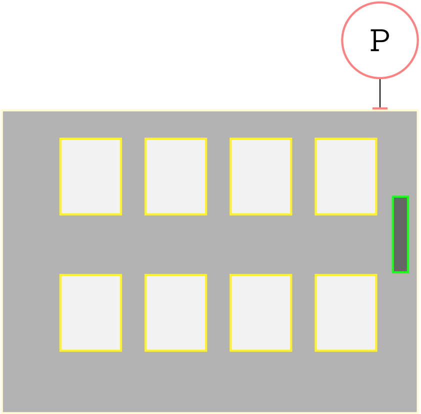
satisfy  ?

Does  satisfy 

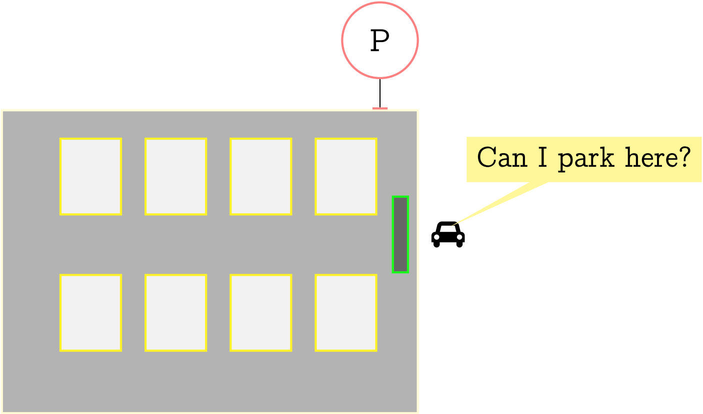
?

Running example: Unbounded Client-Server System (UCS)

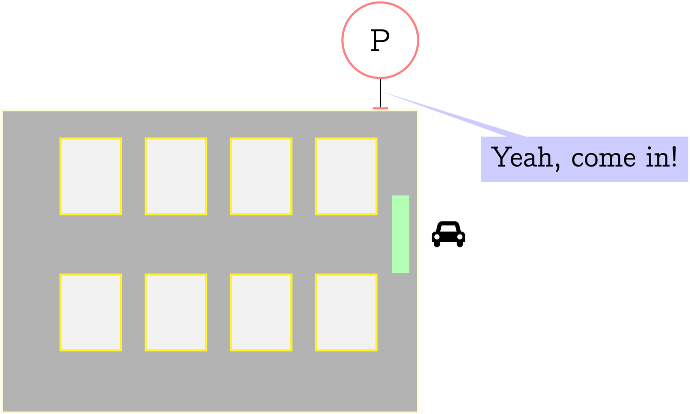
An empty Parking Lot waits for vehicles



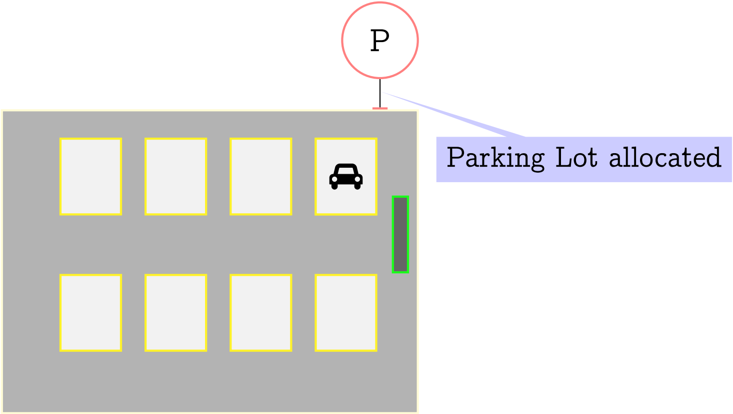
Running example: Unbounded Client-Server System (UCS)



Running example: Unbounded Client-Server System (UCS)

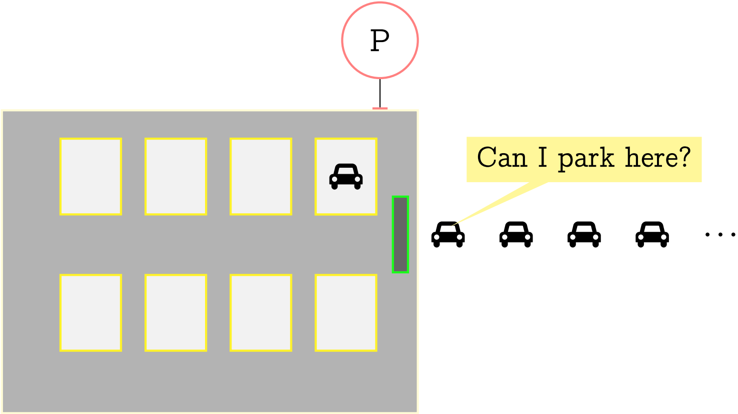


Running example: Unbounded Client-Server System (UCS)



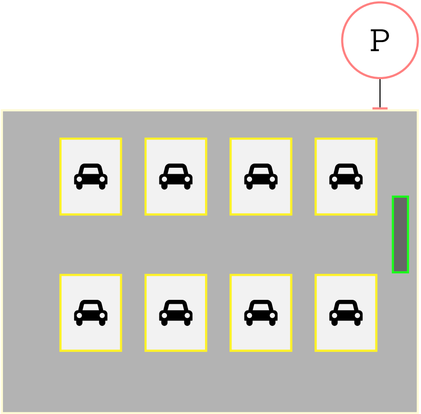
Running example: Unbounded Client-Server System (UCS)

Unbounded number of parking requests

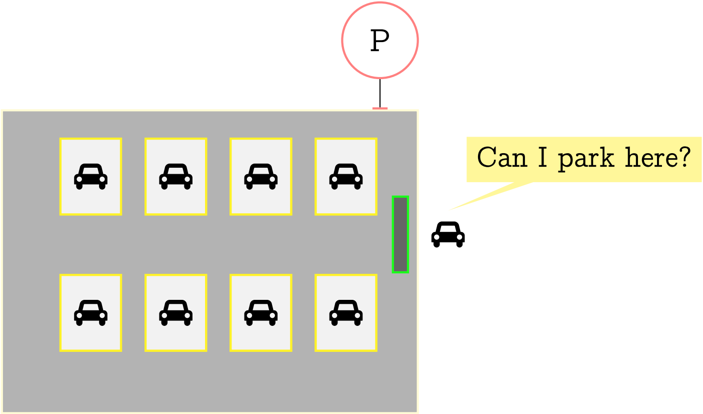


Running example: Unbounded Client-Server System (UCS)

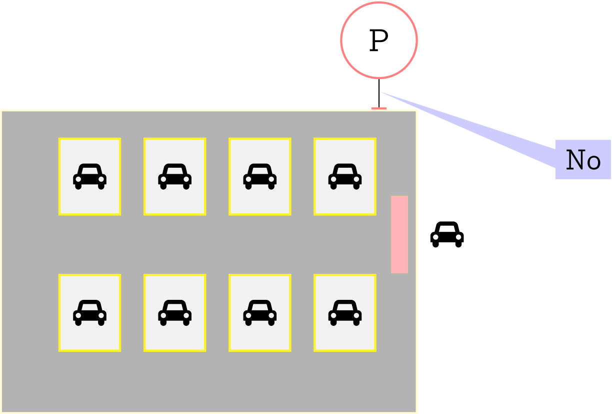
No parking space



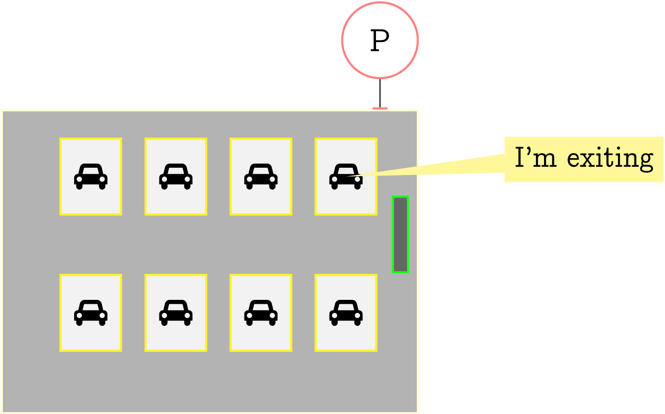
Running example: Unbounded Client-Server System (UCS)



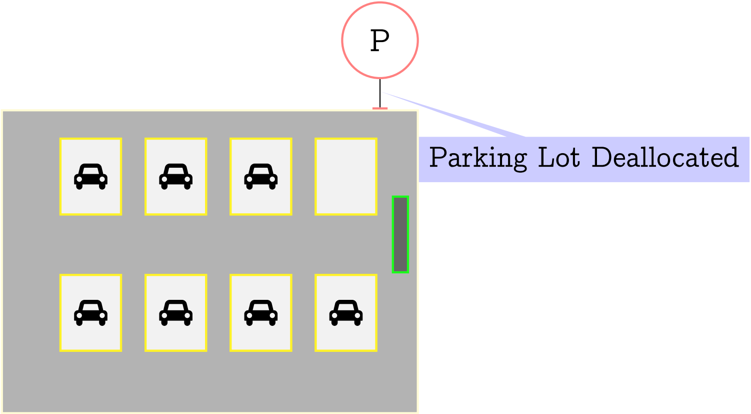
Running example: Unbounded Client-Server System (UCS)



Running example: Unbounded Client-Server System (UCS)

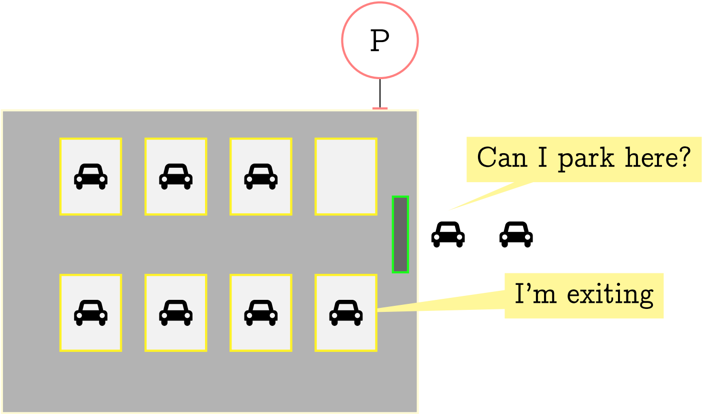


Running example: Unbounded Client-Server System (UCS)

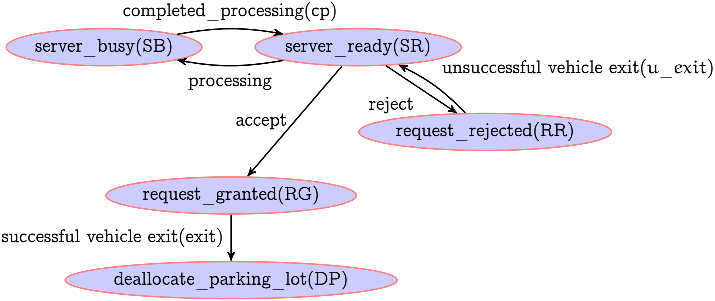


Running example: Unbounded Client-Server System (UCS)

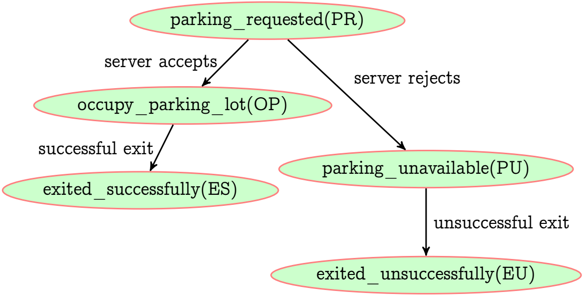
- single server-multiple client system
- unbounded clients of the same type



Case Study Parking System: Server behaviour



Case Study Parking System: Client behaviour



Research Questions

1. How do we formally verify UCS?
2. How do we formally model UCS?
3. How do we express properties of client-server systems naturally?
    1. Do existing logics suffice?
    2. What additional properties are interesting in the context of unbounded client-server systems?

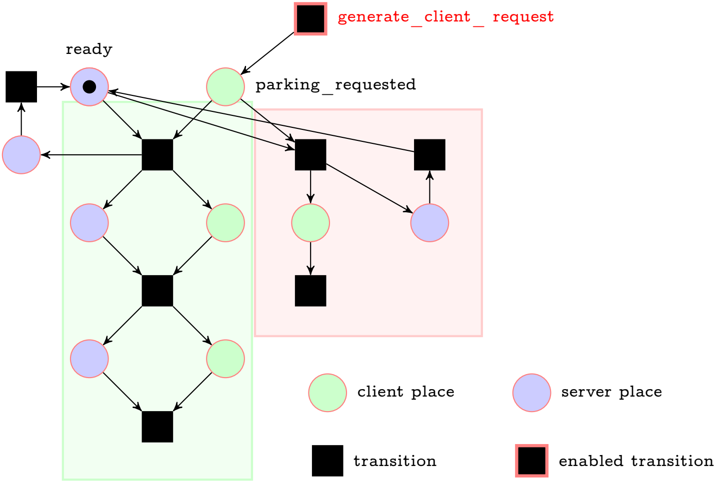
Research Questions

1. How do we formally verify UCS?
2. How do we formally model UCS?
3. How do we express properties of client-server systems naturally?
    1. Do existing logics suffice?
    2. What additional properties are interesting in the context of unbounded client-server systems?
4. Are there formal verification tools for UCS? Can we add to the repertoire?
    1. How are existing tools different, unique? Is another tool necessary?

Challenge 1: Modelling UCS using Petri Nets

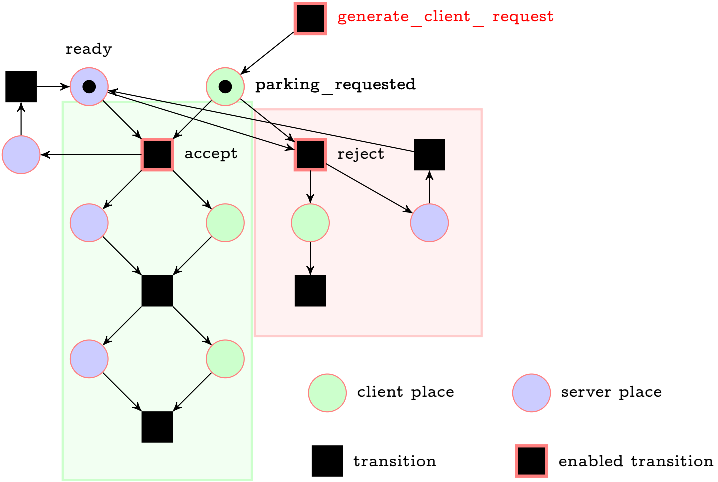
Initially the server is

ready for client requests



Challenge 1: Modelling UCS using Petri Nets

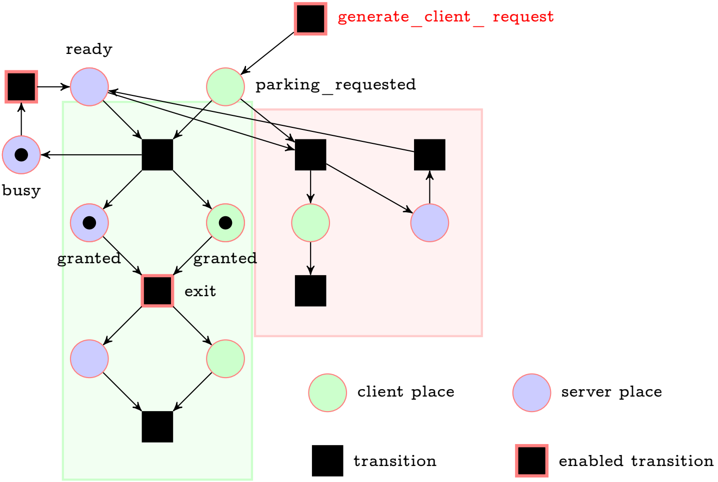
Client parking request is received



Challenge 1: Modelling UCS using Petri Nets

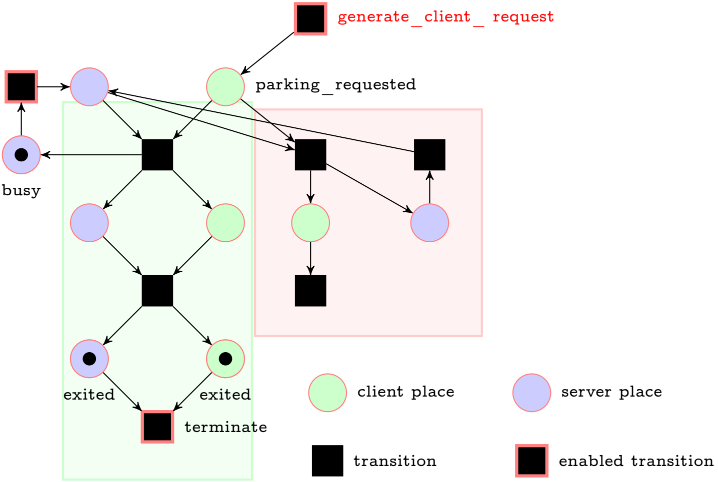
Server accepts the Client

parking request



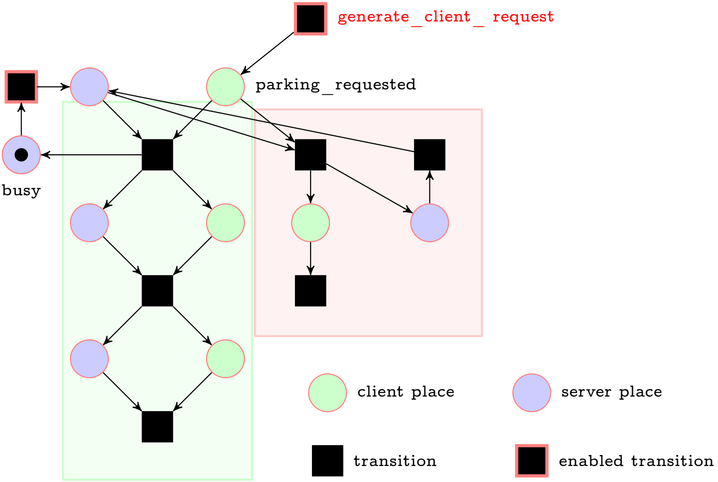
Challenge 1: Modelling UCS using Petri Nets

Client exits the parking space



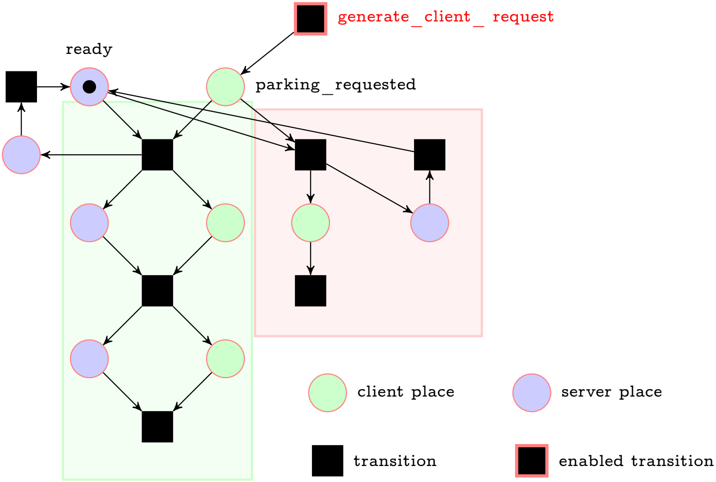
Challenge 1: Modelling UCS using Petri Nets

Client is terminated



Challenge 1: Modelling UCS using Petri Nets

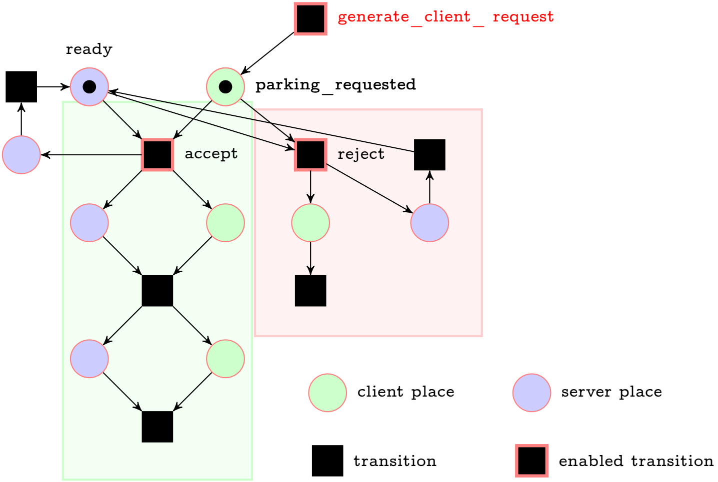
Server is ready for more requests



Challenge 1: Modelling UCS using Petri Nets

A new Client parking

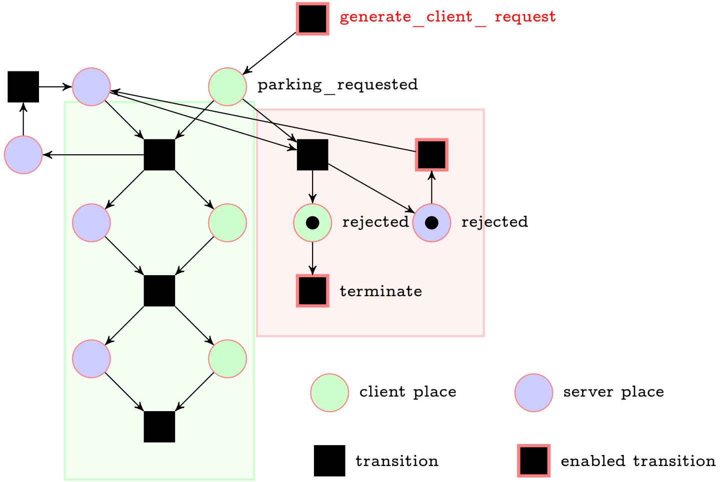
request is received



Challenge 1: Modelling UCS using Petri Nets

Server rejects the Client

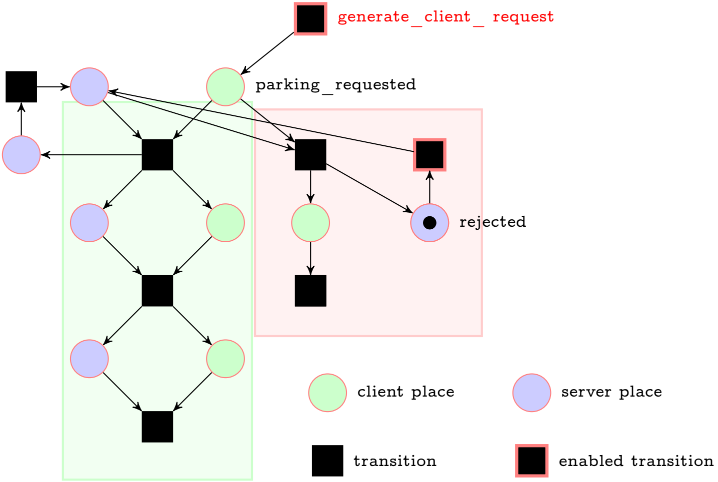
parking request



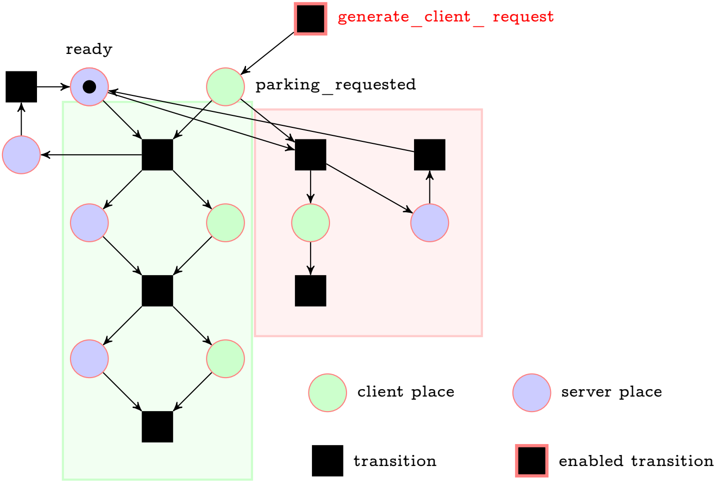
Challenge 1: Modelling UCS using Petri Nets

Client is terminated

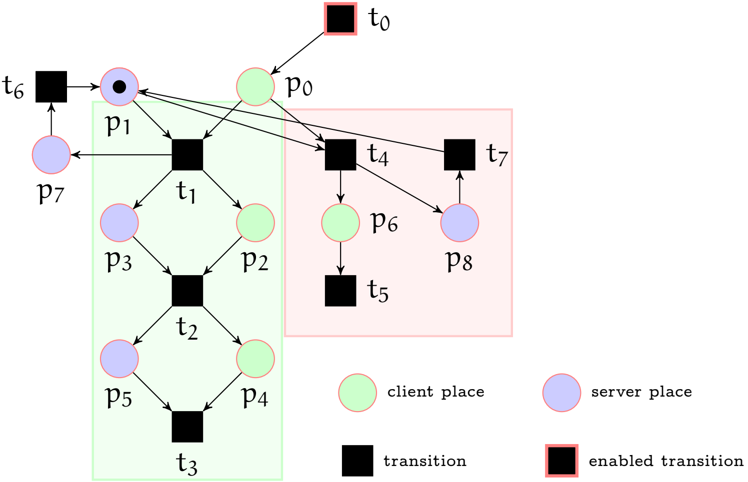
unsuccessfully



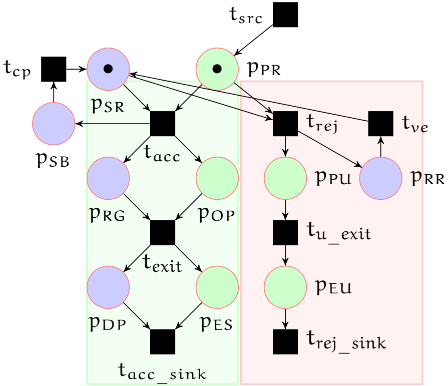
Challenge 1: Modelling UCS using Petri Nets



Challenge 1: Modelling UCS using Petri Nets



Challenge 1: Modelling UCS using Petri Nets



Let t_c be the set containing all token counters.

$$t_c = \{\#p_{PR}, \#p_{OP}, \#p_{PU}, \#p_{ES}, \#p_{EU}, \#p_{SR}, \#p_{SB}, \#p_{RG}, \#p_{RR}, \#p_{DP}\}$$

Question: What properties of the system are we interested in?

Properties

1. Is the number of clients occupying parking lots = number of requests granted always?

   $G(\#p_{OP} = \#p_{RG})$

Properties

1. Is the number of clients occupying parking lots = number of requests granted always?

   $G(\#p_{OP} = \#p_{RG})$

2. Consider the property, $p_{PR} + p_{OP} + p_{ES} = 1$, which is an invariant.

   $G(\#p_{PR} + \#p_{OP} + \#p_{ES} = 1)$.

Properties

1. Is the number of clients occupying parking lots = number of requests granted always?

   $G(\#p_{OP} = \#p_{RG})$

2. Consider the property, $p_{PR} + p_{OP} + p_{ES} = 1$, which is an invariant.

   $G(\#p_{PR} + \#p_{OP} + \#p_{ES} = 1)$.

3. Is the number of rejected requests greater than the number of accepted requests at some point?

   $F(\#p_{EU} > \#p_{ES})$.

Linear Temporal Logic with Integer Arithmetic $\mathrm{LTL}_{\mathrm{LIA}}$

In $\mathrm{LTL}_{\mathrm{LIA}}$, at the propositional level, we allow token counters and arithmetic operators over them.
Let t_c be the set containing all token counters. The formulas in $\mathrm{LTL}_{\mathrm{LIA}}$ are given as follows:

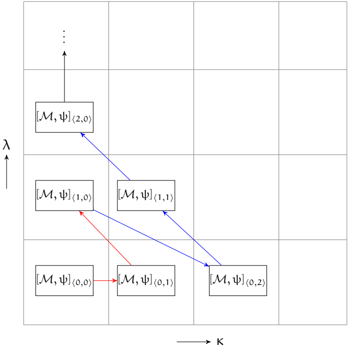
$$\alpha, \hat{\alpha} ::= \#p \mid \mathfrak{c} \mid (\mathfrak{c} * \#p) \mid (\#p * \mathfrak{c}) \mid (\alpha + \hat{\alpha}) \mid (\alpha - \hat{\alpha})$$
$$\beta ::= (\alpha < \hat{\alpha}) \mid (\alpha > \hat{\alpha}) \mid (\alpha \leq \hat{\alpha}) \mid (\alpha \geq \hat{\alpha}) \mid (\alpha = \hat{\alpha})$$

where $\#p \in t_c$ and $\mathfrak{c} \in \mathbb{Z}^+$.

$$\psi ::= \beta \mid \neg\psi \mid \psi \vee \psi' \mid \psi \wedge \psi' \mid X\psi \mid F\psi \mid G\psi \mid \psi U \psi'$$

Here, the modalities X, F, G and U have the same meaning as the LTL temporal operators Next, Eventually, Globally and Until, respectively.

BMC vs 2D-BMC



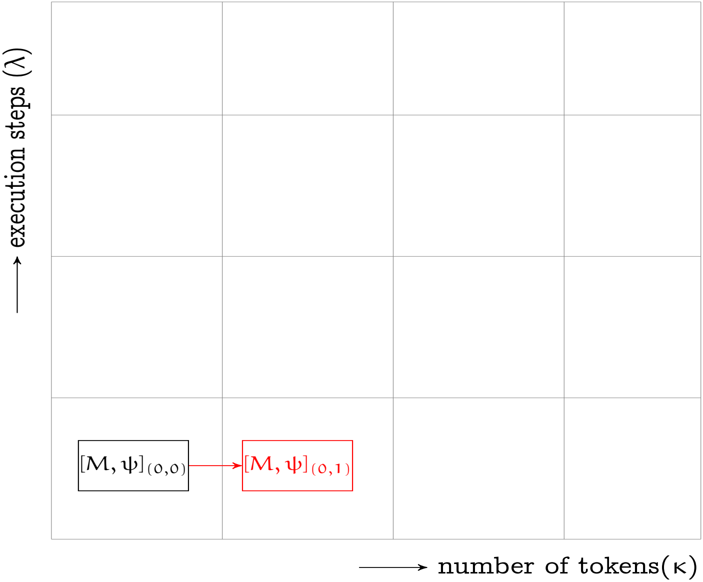
- In BMC, "We ask whether the system M has any counterexample of length k to (property) ψ. This bounded problem is encoded into SAT." - A.Biere[a]
- In 2D-BMC, we ask whether the system M has any counterexample to ψ, of length λ and number of tokens κ and encode this bounded problem into SMT.

[a] Linear Encodings of Bounded LTL Model Checking, Biere et al, LMCS, 2006

The 2D-BMC strategy

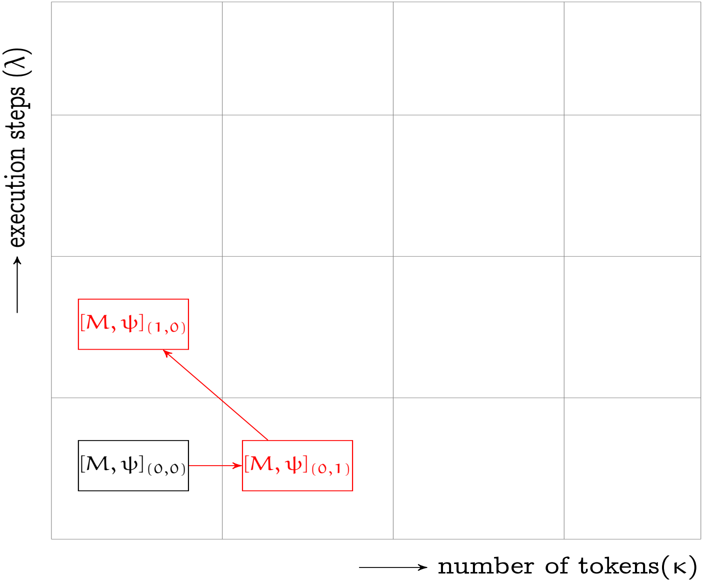
execution steps + number of clients = bound

The 2D-BMC strategy



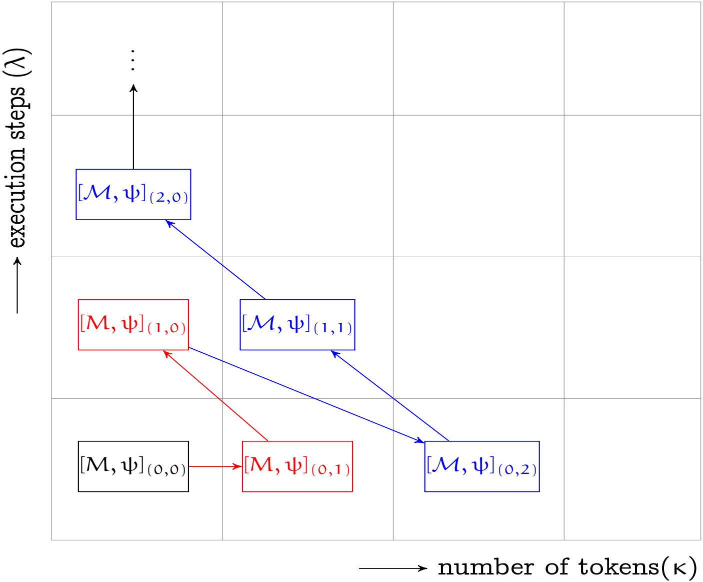
execution steps + number of clients = bound

The 2D-BMC strategy



execution steps + number of clients = bound

The 2D-BMC strategy



execution steps + number of clients = bound

PN Semantics

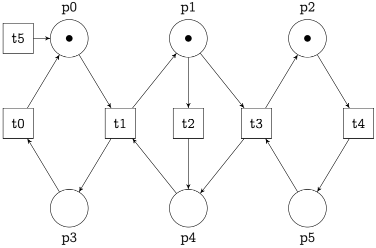


Figure: An unbounded net

$$\begin{bmatrix}1\\1\\1\\0\\0\\0\end{bmatrix} \xrightarrow{t_2} \begin{bmatrix}1\\0\\1\\0\\1\\0\end{bmatrix} \xrightarrow{t_1} \begin{bmatrix}0\\1\\1\\1\\0\\0\end{bmatrix} \xrightarrow{t_4} \begin{bmatrix}0\\1\\0\\1\\0\\1\end{bmatrix} \xrightarrow{t_3} \begin{bmatrix}0\\0\\1\\1\\1\\0\end{bmatrix} \xrightarrow{t_4} \begin{bmatrix}0\\0\\0\\1\\1\\1\end{bmatrix}$$

Figure: Sequence of firings via interleaving semantics

$$\begin{bmatrix}1\\1\\1\\0\\0\\0\end{bmatrix} \xrightarrow{t_2,t_4} \begin{bmatrix}1\\0\\0\\0\\1\\1\end{bmatrix} \xrightarrow{t_1} \begin{bmatrix}0\\1\\0\\1\\0\\1\end{bmatrix} \xrightarrow{t_2} \begin{bmatrix}0\\0\\0\\1\\1\\1\end{bmatrix}$$

Figure: Sequence of firings via concurrent semantics

Tool Architecture

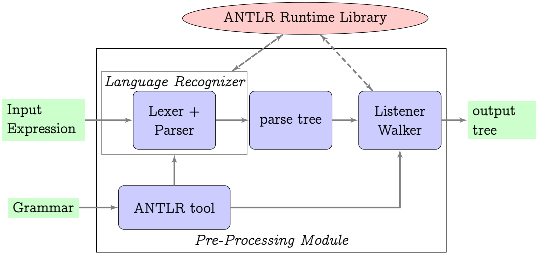


Figure: Pre-Processing the model and formula using ANTLR

Tool Architecture

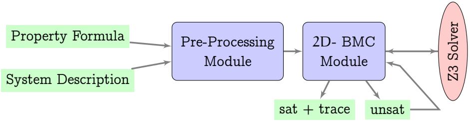


Figure: DCModelChecker architecture

1. DCModelChecker1.0 verifies PN over $\mathrm{LTL}_{\mathrm{LIA}}$ with interleaved semantics `https://doi.org/10.5281/zenodo.7084996`
2. DCModelChecker2.0 verifies PN over $\mathrm{LTL}_{\mathrm{LIA}}$ allowing concurrent semantics `https://doi.org/10.5281/zenodo.7198485`

Experimental Results

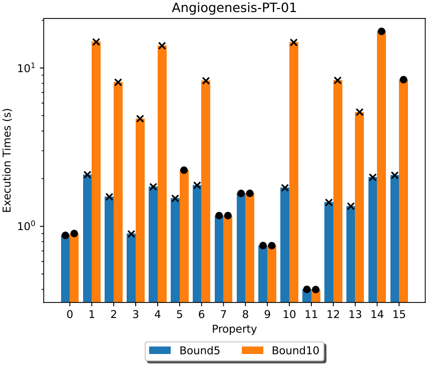


Figure: Verification of FireabilityCardinality Properties with bound 5 and 10

Verified against 16 Benchmarks from Model Checking Contest 2025

Experimental Results

Model	Execution Times(ms)			
	DCModelChecker 1.0	DCModelChecker 2.0	ITS-Tools	Tapaal
Parity	**0.010**	0.40	0.933	0.02
PGCD	**0.019**	0.98	1.487	0.07
Process	0.033	0.85	1.201	**1e-05**
CryptoMiner	0.036	0.64	0.957	**7e-06**
Murphy	0.031	0.83	1.161	**8e-06**

Table: Results of Comparative Experiments on Unbounded PNs

Contributions

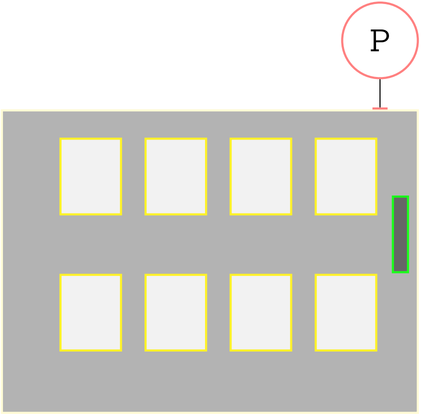
- Part 1: Verification of Unbounded Client-Server (UCS) using Petri Nets and Linear Temporal Logic with integer arithmetic using 2D-BMC

Challenge 2: Modelling an UCS with distinguishable clients

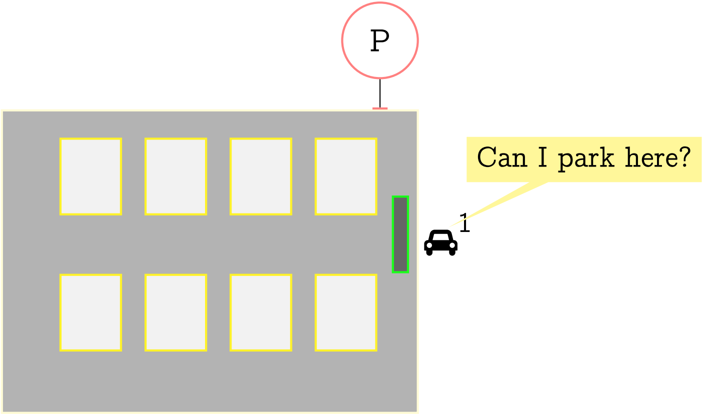
What if clients needed to be distinguished?

Running example

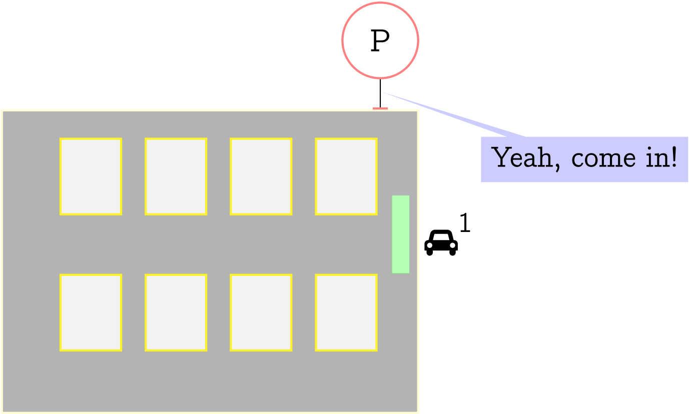
An empty Parking Lot waits for vehicles



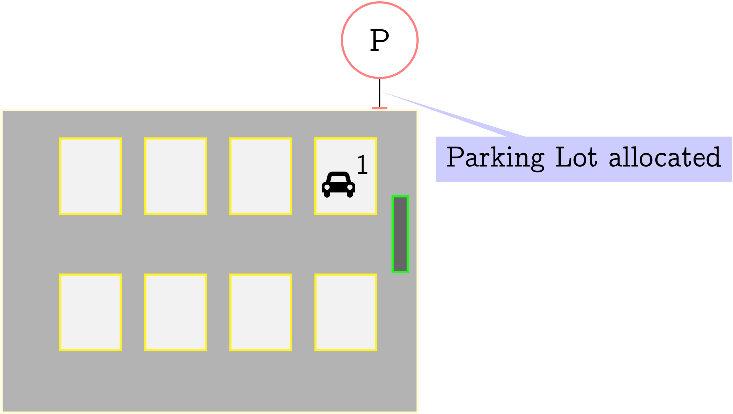
Running example



Running example

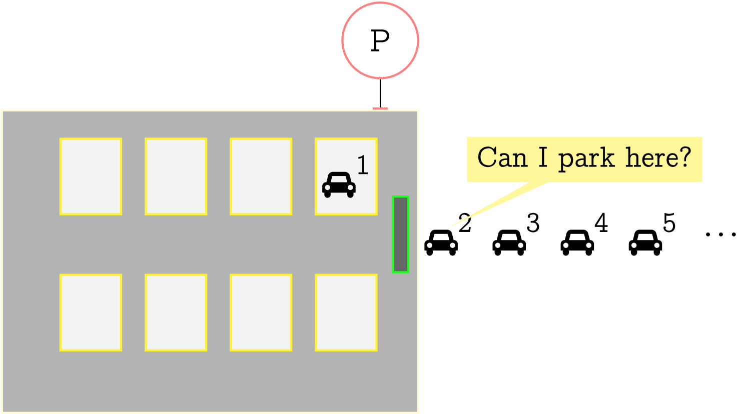


Running example



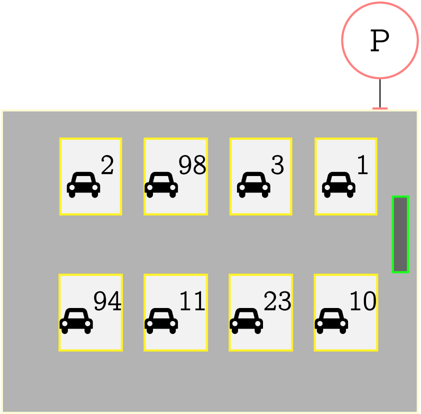
Running example

Unbounded number of parking requests

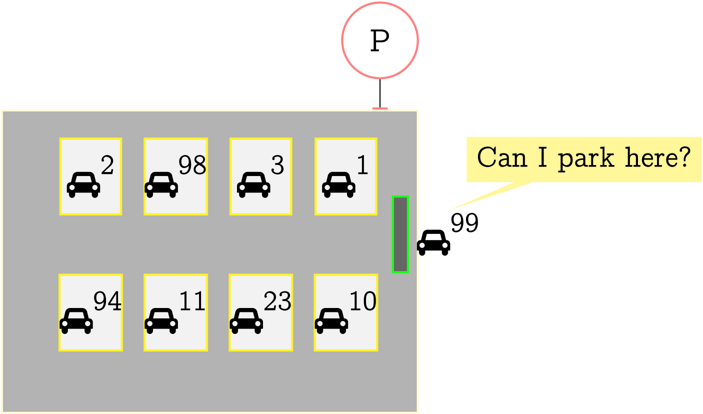


Running example

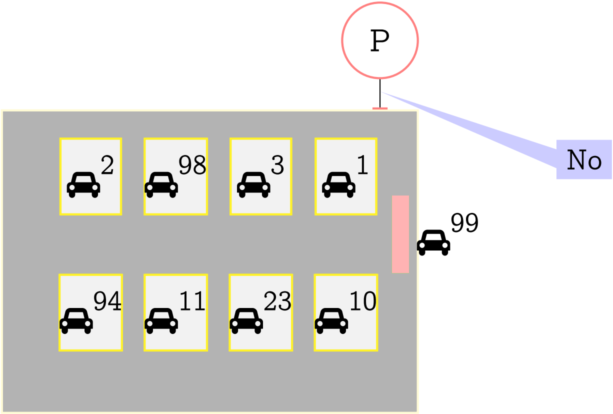
No parking space



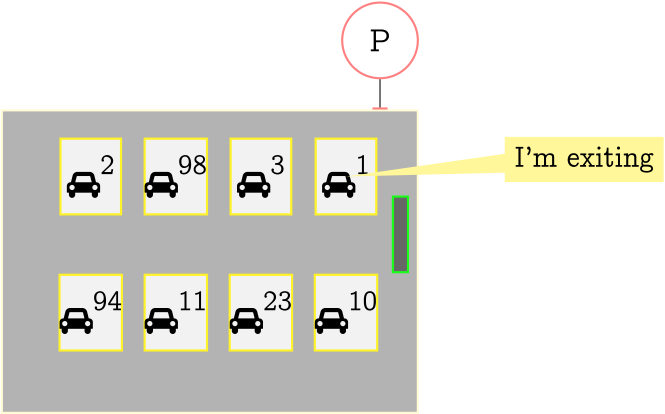
Running example



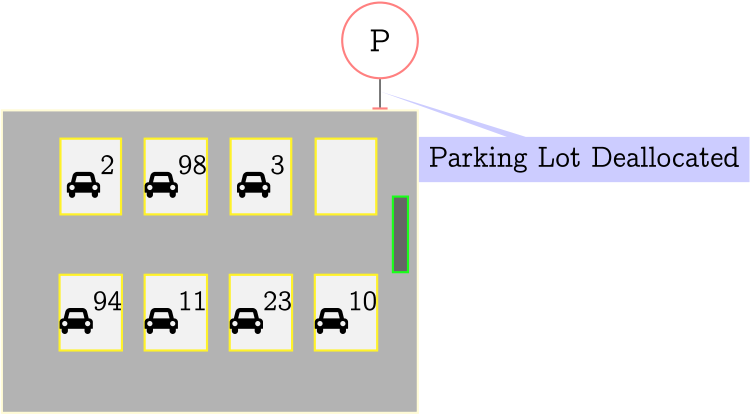
Running example



Running example

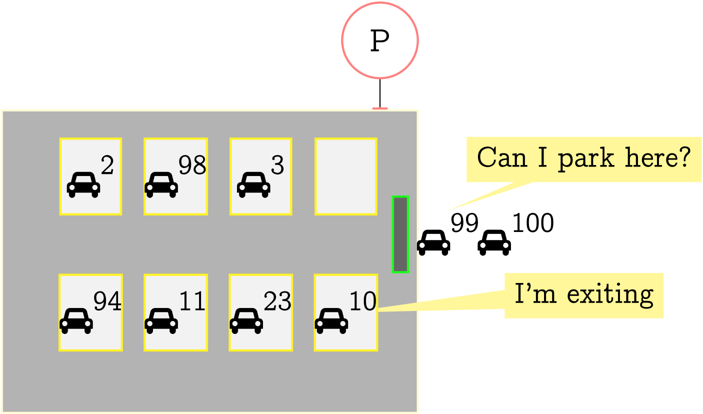


Running example

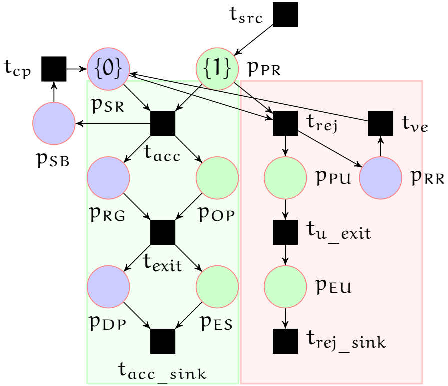


Running example

- single server-multiple client system
- unbounded distinguishable clients of the same type



Modelling an Unbounded Client-Server (UCS)*



Finite sets of atomic client propositions P_c and server propositions P_s:

$P_c = \{$parking_requested(PR),
occupy_parking_lot(OP),
parking_unavailable(PU),
exit_successfully(ES),
exited_unsuccessfully(EU)$\}$

$P_s = \{$server_ready(SR),
server_busy(SB),
request_granted(RG),
request_rejected(RR),
deallocate_parking_lot(DP)$\}$

Question: What properties of the system are we interested in?

Properties

1. When a vehicle requests a parking space, it is always the case that for every vehicle, it eventually exits the system, either successfully after being granted a parking space, or unsuccessfully, when its request is denied.

Properties

1. When a vehicle requests a parking space, it is always the case that for every vehicle, it eventually exits the system, either successfully after being granted a parking space, or unsuccessfully, when its request is denied.

$$G_s(\forall x)\Big(parking\_requested(x) \Rightarrow$$
$$F_c\ (exit\_successfully(x)\ \vee\ exit\_unsuccessfully(x))\Big)$$

Properties

1. When a vehicle requests a parking space, it is always the case that for every vehicle, it eventually exits the system, either successfully after being granted a parking space, or unsuccessfully, when its request is denied.

$$G_s(\forall x)\Big(parking\_requested(x) \Rightarrow$$
$$F_c\ (exit\_successfully(x)\ \vee\ exit\_unsuccessfully(x))\Big)$$

2. It is always the case that if the client occupies a parking lot, it will eventually exit the parking lot.

Properties

1. When a vehicle requests a parking space, it is always the case that for every vehicle, it eventually exits the system, either successfully after being granted a parking space, or unsuccessfully, when its request is denied.

$$G_s(\forall x)\Big(parking\_requested(x) \Rightarrow$$
$$F_c\ (exit\_successfully(x)\ \vee\ exit\_unsuccessfully(x))\Big)$$

2. It is always the case that if the client occupies a parking lot, it will eventually exit the parking lot.

$$G_s(\forall x)\Big(occupy\_parking\_lot(x) \Rightarrow F_c(exit\_successfully(x))\Big)$$

Properties

3. There may be clients whose requests are rejected.

Properties

3. There may be clients whose requests are rejected.

$$G_s(\exists x)\big(parking\_requested(x) \wedge F_c(exit\_unsuccessfully(x))\big)$$

Properties

3. There may be clients whose requests are rejected.

   $$G_s(\exists x)\big(parking\_requested(x) \wedge F_c(exit\_unsuccessfully(x))\big)$$

4. There may be clients who have requested for parking and who wait in the parking unavailable state until they are able to exit the system.

Properties

3. There may be clients whose requests are rejected.

$$G_s(\exists x)\big(parking\_requested(x) \wedge F_c(exit\_unsuccessfully(x))\big)$$

4. There may be clients who have requested for parking and who wait in the parking unavailable state until they are able to exit the system.

$$G_s(\exists x)\Big(parking\_requested(x) \wedge$$
$$F_c\big(parking\_unavailable(x)\ U_c\ exit\_unsuccessfully(x)\big)\Big)$$

The proposed logic: The Monodic[†] Logic $FOTL_1$

The set of client formulae Δ:

$$\Delta ::= p(x) \mid \neg\alpha \mid \alpha \vee \beta \mid \alpha \wedge \beta \mid X_c\alpha \mid F_c\alpha \mid G_c\alpha \mid \alpha\ U_c\ \beta$$

where $\alpha, \beta \in \Delta$ and $p \in P_c$, the set of atomic client propositions.

The proposed logic: The Monodic[†] Logic $FOTL_1$

The set of client formulae Δ:

$$\Delta ::= p(x) \mid \neg\alpha \mid \alpha \vee \beta \mid \alpha \wedge \beta \mid X_c\alpha \mid F_c\alpha \mid G_c\alpha \mid \alpha\ U_c\ \beta$$

where $\alpha, \beta \in \Delta$ and $p \in P_c$, the set of atomic client propositions.

The set of server formulae Ψ:

$$\Psi ::= q \mid \neg\psi \mid (\exists x)\alpha \mid (\forall x)\alpha \mid \psi_1 \vee \psi_2 \mid \psi_1 \wedge \psi_2 \mid X_s\psi \mid F_s\psi \mid G_s\psi \mid \psi_1\ U_s\ \psi_2$$

where $\psi, \psi_1, \psi_2 \in \Psi$ and $q \in P_s$, the set of atomic server propositions, $\alpha \in \Delta$.

The Monodic Logic FOTL_1 (contd)

$$\psi = (\exists x)(\exists y)\Big(\big(PR(x) \wedge PR(y)\big) \wedge F_c\big(ES(x) \wedge ES(y)\big)\Big)$$

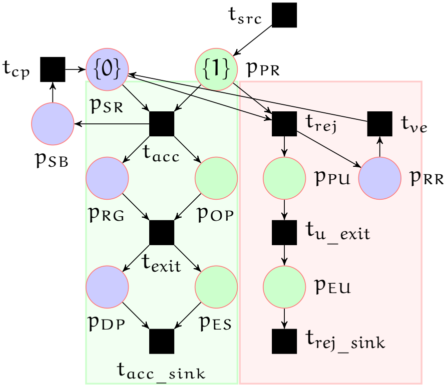
- Is $\psi \in \mathsf{FOTL}_1$?

The Monodic Logic FOTL_1 (contd)

$$\psi = (\exists x)(\exists y)\Big(\big(PR(x) \wedge PR(y)\big) \wedge F_c\big(ES(x) \wedge ES(y)\big)\Big)$$

- Is $\psi \in \mathsf{FOTL}_1$?
- Answer: No. **not a well formed formula** in FOTL_1

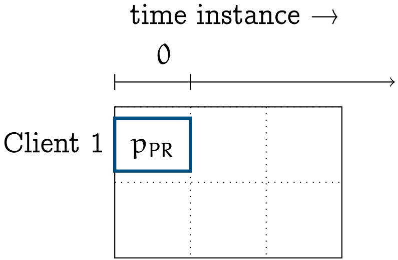
Recall: our model



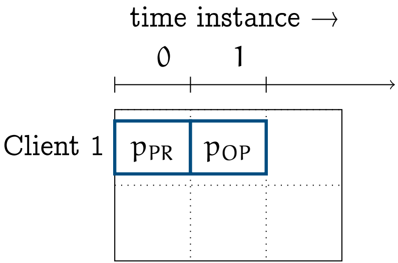
Atomic client propositions P_c and server propositions P_s:

$$
\begin{aligned}
P_c = \{&\mathit{parking\_requested}(PR),\\
&\mathit{occupy\_parking\_lot}(OP),\\
&\mathit{parking\_unavailable}(PU),\\
&\mathit{exit\_successfully}(ES),\\
&\mathit{exited\_unsuccessfully}(EU)\}\\
P_s = \{&\mathit{server\_ready}(SR),\\
&\mathit{server\_busy}(SB),\\
&\mathit{request\_granted}(RG),\\
&\mathit{request\_rejected}(RR),\\
&\mathit{deallocate\_parking\_lot}(DP)\}
\end{aligned}
$$

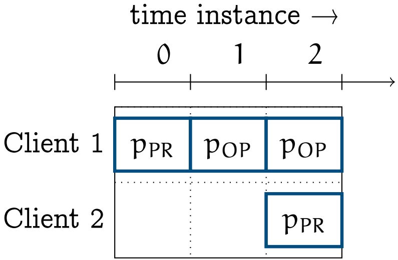
Snapshots of the system + live windows



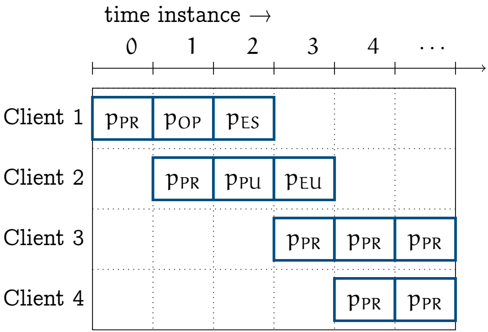
Snapshots of the system + live windows



Snapshots of the system + live windows



Snapshots of the system + live windows (contd)



- 4 distinguishable clients, with overlapping *live windows*, with the bound 5.
- client 1 is at p_{PR} at instance 0. i.e, $PR(x)$ is satisfied in client 1 at instance 0.
- For client 1, the *left boundary* is at instance 0 and its *right boundary* is at instance 2.

Formally speaking...

- Let CN be a countable set of client names.

Formally speaking...

- Let CN be a countable set of client names.
- Let $CS = (Q, \Sigma, \delta, I, F)$ be a finite state machine describing the client behaviour.

Formally speaking...

- Let CN be a countable set of client names.
- Let $CS = (Q, \Sigma, \delta, I, F)$ be a finite state machine describing the client behaviour.
- Let $L : Q \rightarrow 2^{P_c}$ be the labelling function of each client state $q \in Q$ in terms of a subset of propositions P_c true in that state.

Formally speaking...

- Let CN be a countable set of client names.
- Let $CS = (Q, \Sigma, \delta, I, F)$ be a finite state machine describing the client behaviour.
- Let $L : Q \to 2^{P_c}$ be the labelling function of each client state $q \in Q$ in terms of a subset of propositions P_c true in that state.
- Let $\mathfrak{Z} : CN \times \mathbb{N}_0 \to Q$ be a partial mapping describing the local state of each client $a \in CN$ at an instance $i \in \mathbb{N}_0$.

Formally speaking...

- Let CN be a countable set of client names.
- Let $CS = (Q, \Sigma, \delta, I, F)$ be a finite state machine describing the client behaviour.
- Let $L : Q \to 2^{P_c}$ be the labelling function of each client state $q \in Q$ in terms of a subset of propositions P_c true in that state.
- Let $\mathfrak{Z} : CN \times \mathbb{N}_0 \to Q$ be a partial mapping describing the local state of each client $a \in CN$ at an instance $i \in \mathbb{N}_0$.
  - For instance, $\mathfrak{Z}(a, i) \in q$, means that the local state of each client a at instance i is state q, where $q \in Q$.

Formally speaking...

A model is a triple $M = (\nu, V, \xi)$ where

- ν gives the local behaviour of the server as follows:
  $\nu = \nu_0\nu_1\nu_2\ldots$, where for all $0 \leq i$, $\nu_i \subseteq P_s$

Formally speaking...

A model is a triple $M = (\nu, V, \xi)$ where

- ν gives the local behaviour of the server as follows:
  $\nu = \nu_0\nu_1\nu_2 \ldots$, where for all $0 \leq i$, $\nu_i \subseteq P_s$
- V gives the set of live agents (clients) at each instance.
  $V = V_0V_1V_2 \ldots$, where for all $0 \leq i$, V_i is a finite subset of CN, gives the set of live agents at the ith instance.
  For every $0 \leq i$, V_i and V_{i+1} satisfy the following properties:
  1. if $V_i \subseteq V_{i+1}$ then for every $a \in V_{i+1} - V_i$ such that $\mathfrak{Z}(a, i+1) \in I$.
  2. if $V_{i+1} \subseteq V_i$ then for every $a \in V_i - V_{i+1}$ such that $\mathfrak{Z}(a, i) \in F$.

Formally speaking...

A model is a triple $M = (\nu, V, \xi)$ where

- ν gives the local behaviour of the server as follows: $\nu = \nu_0\nu_1\nu_2\ldots$, where for all $0 \le i$, $\nu_i \subseteq P_s$
- V gives the set of live agents (clients) at each instance. $V = V_0V_1V_2\ldots$, where for all $0 \le i$, V_i is a finite subset of CN, gives the set of live agents at the ith instance. For every $0 \le i$, V_i and V_{i+1} satisfy the following properties:
  1. if $V_i \subseteq V_{i+1}$ then for every $a \in V_{i+1} - V_i$ such that $\mathfrak{Z}(a, i+1) \in I$.
  2. if $V_{i+1} \subseteq V_i$ then for every $a \in V_i - V_{i+1}$ such that $\mathfrak{Z}(a, i) \in F$.
- $\xi = \xi_0\xi_1\xi_2\cdots$, where for all $0 \le i$, $\xi_i : V_i \to 2^{P_c}$ gives the properties satisfied by each live agent at ith instance. Also, $L(\mathfrak{Z}(a, i)) = \xi_i(a)$.

Semantics

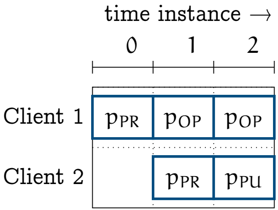
$M, i \models (\exists x)\alpha$ iff $\exists a \in CN,\ a \in V_i$ and $M, [x \mapsto a], i \models_x \alpha$.

Semantics

$M, i \models (\exists x)\alpha$ iff $\exists a \in CN$, $a \in V_i$ and $M, [x \mapsto a], i \models_x \alpha$.
$M, i \models^k (\exists x)\alpha$ iff $\exists a \in CN$, $a \in V_i$ and $M, [x \mapsto a], i \models_x^k \alpha$.

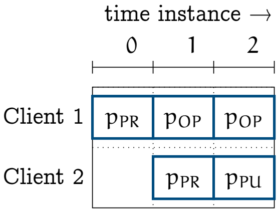
Semantics

$M, i \models (\exists x)\alpha$ iff $\exists a \in CN$, $a \in V_i$ and $M, [x \mapsto a], i \models_x \alpha$.
$M, i \models^k (\exists x)\alpha$ iff $\exists a \in CN$, $a \in V_i$ and $M, [x \mapsto a], i \models_x^k \alpha$.



Semantics

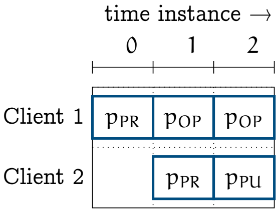
$M, i \models (\exists x)\alpha$ iff $\exists a \in CN$, $a \in V_i$ and $M, [x \mapsto a], i \models_x \alpha$.
$M, i \models^k (\exists x)\alpha$ iff $\exists a \in CN$, $a \in V_i$ and $M, [x \mapsto a], i \models_x^k \alpha$.



Given $\alpha = OP(x) \vee PR(x)$ and $CN = \{1, 2\}$.

Semantics

$M, i \models (\exists x)\alpha$ iff $\exists a \in CN$, $a \in V_i$ and $M, [x \mapsto a], i \models_x \alpha$.
$M, i \models^k (\exists x)\alpha$ iff $\exists a \in CN$, $a \in V_i$ and $M, [x \mapsto a], i \models_x^k \alpha$.

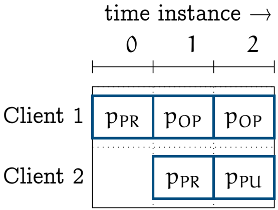


Given $\alpha = OP(x) \vee PR(x)$ and $CN = \{1, 2\}$.

- $M, 0 \models^k (\exists x)(OP(x) \vee PR(x))$.
- $M, 1 \models^k (\exists x)(OP(x) \vee PR(x))$.
- $M, 2 \models^k (\exists x)(OP(x) \vee PR(x))$.

Bounded Semantics

$$M, i \models^k (\forall x)\alpha \text{ iff } \forall a \in CN, \text{ if } a \in V_i \text{ then } M, [x \mapsto a], i \models^k_x \alpha$$



Given $\alpha = ES(x)$ and $CN = \{1, 2\}$.

- $M, 0 \not\models^k (\exists x)(ES(x))$.
- $M, 1 \not\models^k (\exists x)(ES(x))$.
- $M, 2 \not\models^k (\exists x)(ES(x))$.

Bounded Semantics

$M, i \models^k F_s\psi$ iff $\exists j : i \leq j \leq k$, $M, j \models^k \psi$.
Given $\psi = (\exists x)ES(x)$.

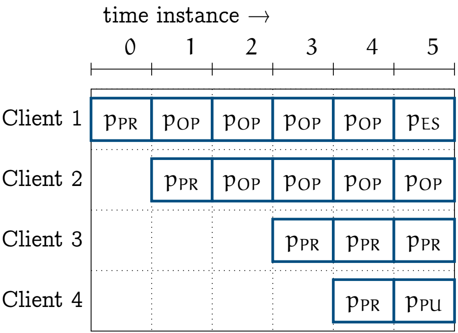


Figure: Snapshot with 4 clients

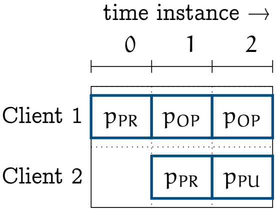


Figure: Snapshot with 2 clients

Bounded Semantics

$M, i \models^k F_s\psi$ iff $\exists j : i \leq j \leq k$, $M, j \models^k \psi$.
Given $\psi = (\exists x)ES(x)$.

time instance →

	0	1	2	3	4	5
Client 1	p_{PR}	p_{OP}	p_{OP}	p_{OP}	p_{OP}	p_{ES}
Client 2		p_{PR}	p_{OP}	p_{OP}	p_{OP}	p_{OP}
Client 3				p_{PR}	p_{PR}	p_{PR}
Client 4					p_{PR}	p_{PU}

Figure: Snapshot with 4 clients

time instance →

	0	1	2
Client 1	p_{PR}	p_{OP}	p_{OP}
Client 2		p_{PR}	p_{PU}

Figure: Snapshot with 2 clients

As shown above $M, 5 \models^k F_s(\exists x)ES(x)$.

Implementation

Verifying $G_s \exists x(F_c(p_0(x)))$

```
 evname: cToken_4_1
 evname: cToken_4_2
 evname: cToken_4_3
 evname: cToken_4_4
 evname: cToken_4_5
 evname: sToken_4_1
LOG: done with initializing empty token variables
LOG: sysObj.MAX_TOKENS_LIMIT before initial marking: 1
LOG: constructed initial Marking
LOG: constructed the complete Transition Function
LOG: nochange constraint added
LOG: UNSAT
LOG: UNSAT
LOG: UNSAT
LOG: UNSAT
LOG: --------------------------run 5 ----------------------------------
 evname: cToken_5_0
 evname: cToken_5_1
 evname: cToken_5_2
 evname: cToken_5_3
 evname: cToken_5_4
 evname: cToken_5_5
 evname: sToken_5_1
LOG: done with initializing empty token variables
LOG: sysObj.MAX_TOKENS_LIMIT before initial marking: 1
LOG: constructed initial Marking
LOG: constructed the complete Transition Function
LOG: nochange constraint added
LOG: UNSAT
LOG: UNSAT
LOG: UNSAT
LOG: UNSAT
LOG: UNSAT
```

Contributions

- Part 1: Verification of UCS using Petri Nets and $\mathsf{LTL}_{\mathrm{LIA}}$
- Part 2: Verification of UCS with distinguishable clients using ν-nets and properties specified in FOTL_1

Challenge 3: Modelling an UCS with internal structure

What if clients had an internal structure?

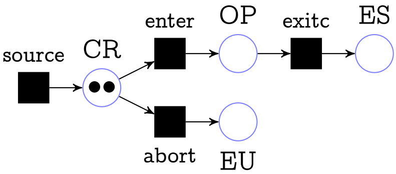


Figure: Petri net N_3 depicting client behaviour

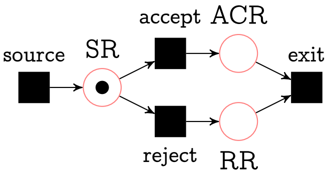


Figure: Petri net N_3 depicting server behaviour

Modelling UCS via Elementary Object Systems

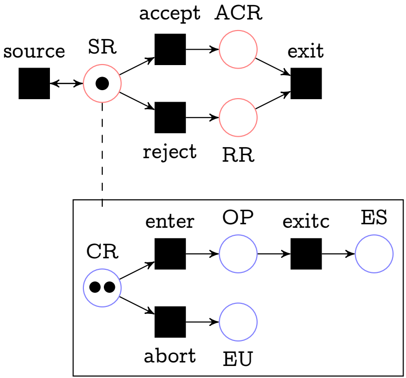


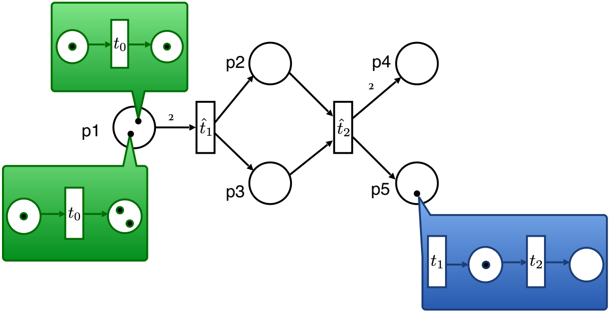
Figure: EOS modeling the single server (unbounded) multiple client system

Question: What properties of the system are we interested in?

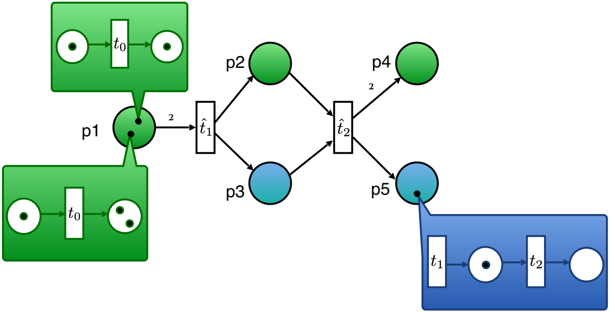
- **Reachability:** Is $\mu_1 = \langle \mathtt{SR}, \{\!\{\mathtt{client1CR}, \mathtt{client2CR}\}\!\} \rangle$ a reachable marking?
- **Coverability:** Given $\mu_1 = \langle \mathtt{SR}, \{\!\{\mathtt{client1CR}, \mathtt{client2OP}\}\!\} \rangle$ and $\mu_2 = \langle \mathtt{SR}, \{\!\{\mathtt{client1OP}, \mathtt{client2CR}, \mathtt{client3OP}\}\!\} \rangle$.

  Then, is $\mu_1 \preceq \mu_2$?

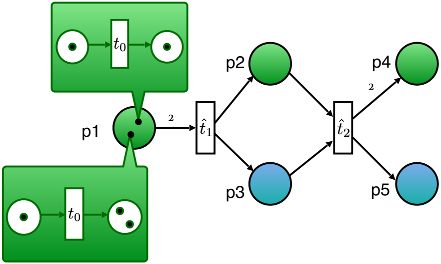
EOS – nested markings



EOS – typed places



EOS – object autonomous events



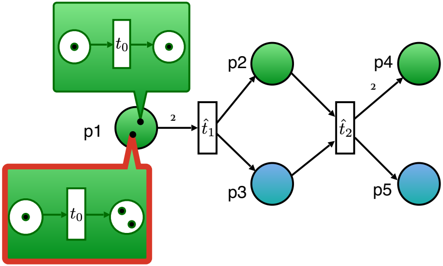
EVENTS

$$e_1 = \left(id_{p_1}, t_0\right)$$

$$e_2 = \left(\hat{t}_1, \emptyset\right)$$

$$e_3 = \left(\hat{t}_2, t_0 t_1 t_1\right)$$

EOS – object autonomous events



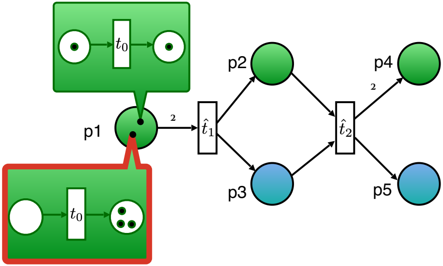
EVENTS

$$e_1 = \left(id_{p_1}, t_0\right)$$

$$e_2 = \left(\hat{t}_1, \varnothing\right)$$

$$e_3 = \left(\hat{t}_2, t_0 t_1 t_1\right)$$

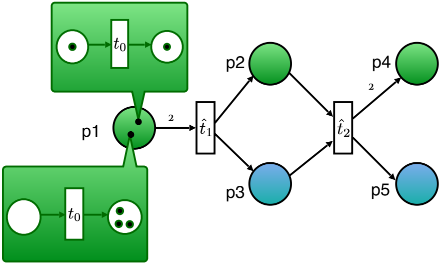
EOS – object autonomous events



EVENTS

$$e_1 = \left(id_{p_1}, t_0\right)$$
$$e_2 = \left(\hat{t}_1, \varnothing\right)$$
$$e_3 = \left(\hat{t}_2, t_0 t_1 t_1\right)$$

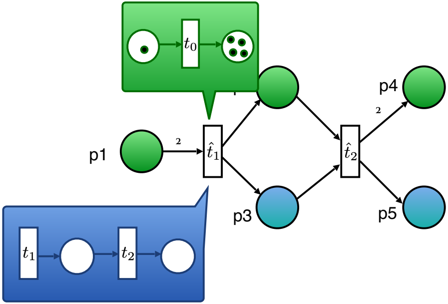
EOS – system autonomous events



EVENTS

$$e_1 = \left(id_{p_1}, t_0\right)$$
$$e_2 = \left(\hat{t}_1, \varnothing\right)$$
$$e_3 = \left(\hat{t}_2, t_0 t_1 t_1\right)$$

EOS – system autonomous events



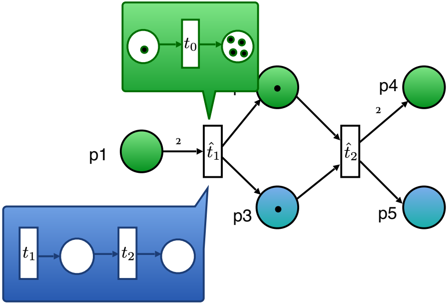
EVENTS

$$e_1 = \left(id_{p_1}, t_0\right)$$

$$e_2 = \left(\hat{t}_1, \varnothing\right)$$

$$e_3 = \left(\hat{t}_2, t_0 t_1 t_1\right)$$

EOS – system autonomous events



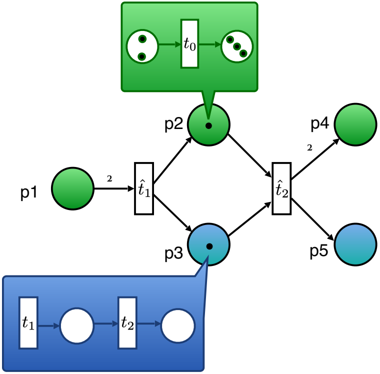
EVENTS

$e_1 = \left(id_{p_1}, t_0\right)$

$e_2 = \left(\hat{t}_1, \varnothing\right)$

$e_3 = \left(\hat{t}_2, t_0 t_1 t_1\right)$

EOS – system autonomous events



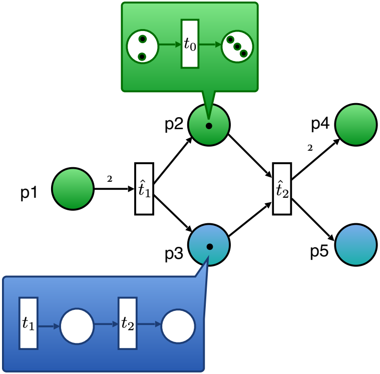
EVENTS

$$e_1 = \left(id_{p_1}, t_0\right)$$

$$e_2 = \left(\hat{t}_1, \varnothing\right)$$

$$e_3 = \left(\hat{t}_2, t_0 t_1 t_1\right)$$

EOS – synchronization events



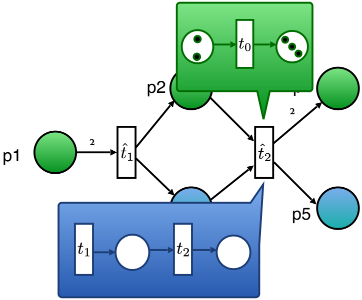
EVENTS

$$e_1 = \left(id_{p_1}, t_0\right)$$

$$e_2 = \left(\hat{t}_1, \varnothing\right)$$

$$e_3 = \left(\hat{t}_2, t_0 t_1 t_1\right)$$

EOS – synchronization events



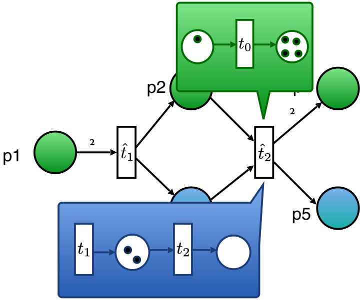
EVENTS

$$e_1 = \left(id_{p_1}, t_0\right)$$

$$e_2 = \left(\hat{t}_1, \varnothing\right)$$

$$e_3 = \left(\hat{t}_2, t_0 t_1 t_1\right)$$

EOS – synchronization events



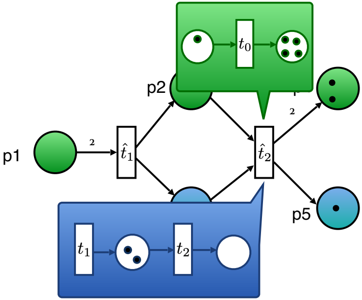
EVENTS

$$e_1 = \left(id_{p_1}, t_0\right)$$

$$e_2 = \left(\hat{t}_1, \varnothing\right)$$

$$e_3 = \left(\hat{t}_2, t_0 t_1 t_1\right)$$

EOS – synchronization events



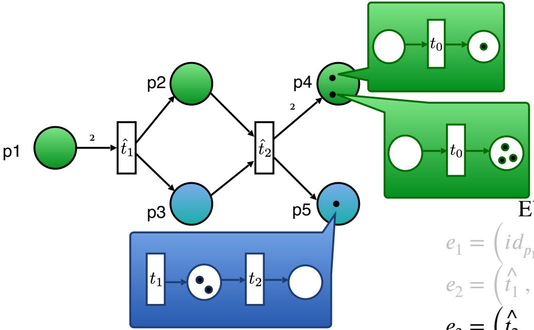
EVENTS

$e_1 = \left(id_{p_1}, t_0\right)$

$e_2 = \left(\hat{t}_1, \varnothing\right)$

$e_3 = \left(\hat{t}_2, t_0 t_1 t_1\right)$

EOS – synchronization events



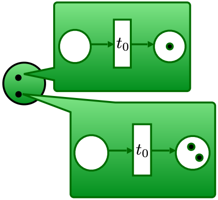
EVENTS

$$e_1 = \left(id_{p_1}, t_0\right)$$

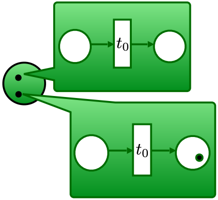
$$e_2 = \left(\hat{t}_1, \varnothing\right)$$

$$e_3 = \left(\hat{t}_2, t_0 t_1 t_1\right)$$

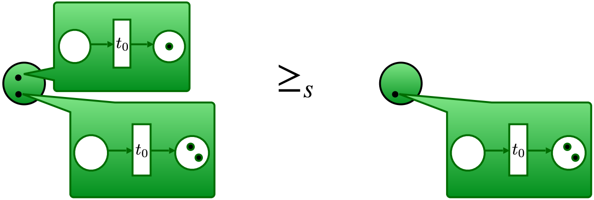
Object lossiness



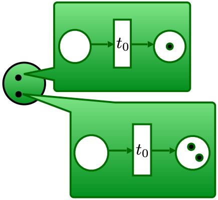
$\geq_o$



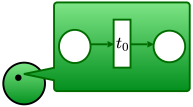
System lossiness



Full lossiness



$$\geq_f$$



$(\preccurlyeq, \ell)$-lossy runs

Perfect runs: only standard steps

$M_1 \to M_2 \to M_3 \to \dots$

$(\preccurlyeq, \ell)$-runs: at most $\ell \leq |\mathbb{N}|$ steps of type $\preccurlyeq$

$M_1 \to M_2 \succcurlyeq M'_3 \to M'_4 \succcurlyeq M'_5 \succcurlyeq M'_6 \to M_7 \to \dots$

$(\preccurlyeq, \ell)$-lossy problems

Is the system **robust** up to ℓ occurrences of $\preccurlyeq$?

$(\preccurlyeq, \ell)$-lossy problems

Is the system **robust** up to ℓ occurrences of $\preccurlyeq$?

$(\preccurlyeq, \ell)$*-deadlock freeness*

Input

An EOS E and an initial marking M.

Output

Is there a $(\preccurlyeq, \ell)$-run from M to a marking where no event is enabled?

Decidability Results (PNSE'24)

	Problems	$\preccurlyeq_f$	$\preccurlyeq_o$	$\preccurlyeq_s$
Conservative EOS	0-reach	**Undec** [Buß14]	**Undec** [Buß14]	**Undec** [Buß14]
	cover	**Dec** [Buß14]	**Undec** (2CM)	**Undec** (2CM)
	ℓ-reach/cover	**Dec** (Comp.)	**Undec** (Comp.)	**Undec** (Comp.)
	ω-reach/cover	**Dec** (Comp.)	**Undec** (Comp.)	**Undec** (Comp.)
EOS	0-reach	**Undec** [Buß14]	**Undec** [Buß14]	**Undec** [Buß14]
	cover	**Undec** [Buß14]	**Undec** (cEOS)	**Undec** (cEOS)
	ℓ-reach/cover	**Undec** (Gadget)	**Undec** (cEOS)	**Undec** (Comp.)
	ω-reach/cover	**Dec** (WSTS)	**Undec** (cEOS)	**Undec** (Comp.)

[Buß14] Köhler-Bußmeier, Michael. 'A Survey of Decidability Results for Elementary Object Systems'. 1 Jan. 2014 : 99 – 123.

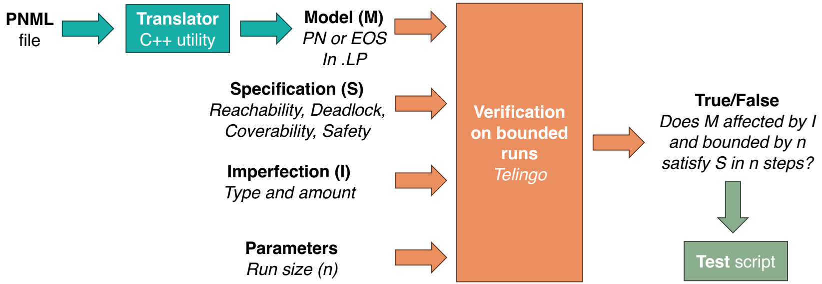
28

Motivation

Current PN tools lack of

- EOS support
- Imperfection support
- Robustness analysis
- Partial verification

Prototype (CILC'24)



Why bounded verification?

Problems on general EOS	$\preccurlyeq_f$	$\preccurlyeq_o$	$\preccurlyeq_s$
0-reach	U	U	U
0-cover	U	U	U
ℓ-reach/cover	U	U	U
ω-reach/cover	D	U	U

F. Di Cosmo, S. Mal, T. Prince, *Deciding Reachability and Coverability in Lossy EOS*, PNSE'24

Why Telingo?

Telingo is **declarative** and supports **temporal constraints**

- E.g., `:- &tel(>? (lossy >(>? lossy)` allows at most one lossy step
- The meaning of lossy is declared orthogonally to EOS specification

Telingo **returns finite runs**

- Perfectly matches bounded verification

Encoding of PNs and EOSs is **elegant in ASP**

- E.g., when compared to SMT – R. Phawade, T. Prince, S. Sheerazuddin et al., *Bounded Model Checking for Unbounded Client Server Systems,* Arxiv (2022)

Correctness and performances

Correct answers

- Checked on MCC benchmarks

Slow on PNs

- Compared with Tapaal

Prohibitively slow on EOSs

- Nesting exacerbates grounding

problem	lossiness	−imax =5 (s)	−imax =10 (s)	−imax =20 (s)	TAPAAL (s)
deadlock	none	UNSAT in 0.052	SAT in 0.622	SAT in 82.754	SAT in $5e-6$
deadlock	any	SAT in 0.009	SAT in 0.010	SAT in 0.009	NA
1-safeness	none	UNSAT in 0.010	UNSAT in 0.014	UNSAT in 0.027	UNSAT in 0
1-safeness	any	UNSAT in 0.013	UNSAT in 0.018	UNSAT in 0.032	NA

Table 1
Comparative Results with TAPAAL for the Eratosthenes-PT-010 PN from the MCC benchmarks [12].

Contributions

- Part 1: Verification of Unbounded Client-Server (UCS) using Petri Nets and Linear Temporal Logic with integer arithmetic
- Part 2: Verification of UCS with distinguishable clients using ν-nets and properties specified in monodic First Order Logic
- Part 3: Verification of UCS where the clients have internal structure using Elementary Object Systems and properties specified in Linear Temporal Logic

List of publications:

1. Ramchandra Phawade, Tephilla Prince, S. Sheerazuddin: Two dimensional bounded model checking for unbounded Petri nets. Arxiv (2026), submitted to TopNOC
2. Ramchandra Phawade, Tephilla Prince, S. Sheerazuddin: Verification of Unbounded Client-Server Systems with Distinguishable Clients. Arxiv (2026), submitted to TopNOC
3. Tephilla Prince: On Verifying Unbounded Client-Server Systems. Springer EUMAS 2023: 465-471
4. Francesco Di Cosmo, Tephilla Prince: Bounded Verification of Petri Nets and EOSs using Telingo: An Experience Report. CILC 2024

List of publications:

5 Francesco Di Cosmo, Soumodev Mal, Tephilla Prince: Deciding Reachability and Coverability in Lossy EOS. PNSE@Petri Nets 2024: 74-95

6 Francesco Di Cosmo, Soumodev Mal, Tephilla Prince: Nets-Within-Nets Through the Lens of Data Nets. Reachability Problems 2025

7 Francesco Di Cosmo, Soumodev Mal, Tephilla Prince: Decidability and Complexity of Reachability and Coverability in Lossy EOS (Yet to be submitted)

List of tools built

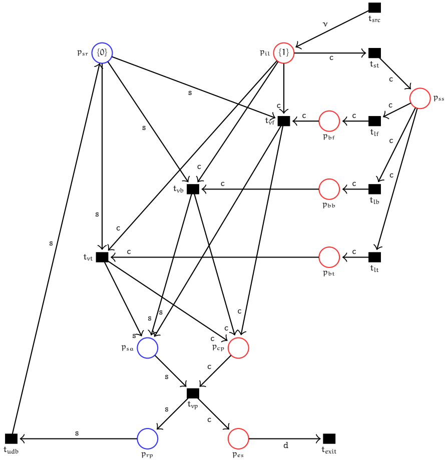
1. DCModelChecker1.0 verifies PN over $\mathrm{LTL}_{\mathrm{LIA}}$ with interleaved semantics
   `https://doi.org/10.5281/zenodo.7084996`
2. DCModelChecker2.0 verifies PN over $\mathrm{LTL}_{\mathrm{LIA}}$ allowing concurrent semantics
   `https://doi.org/10.5281/zenodo.7198485`
3. UCSChecker verifies ν-nets over FOTL_1 properties
   `https://doi.org/10.5281/zenodo.15847737`
4. NWN Telingo Analyser: verifies lossy and perfect PN and EOSs over LTL properties
   `https://doi.org/10.5281/zenodo.11401876`

Appendix

Experimental Results

Our experiments evaluate the following:

1. the correctness of results returned by DCModelChecker 2.0 in comparison with the state-of-the-art ITS-Tools, obtaining tool confidence 0.92.
2. the comparison of DCModelChecker 2.0 on FireabilityCardinality properties which are not verifiable by tools in the MCC for various bounds,
3. verification of unbounded Petri nets in comparison with other tools,
4. the strength of DCModelChecker 2.0 is in additionally verifying invariants specified in $\mathrm{LTL}_{\mathrm{LIA}}$ which are not verifiable by tools in the MCC nor DCModelChecker 1.0.


t_src
v
p_sr {0}
p_il {1}
c
t_st
c
p_ss
s
c
c
c
t_vf
p_bf
t_lf
s
c
c
c
t_vb
c
p_bb
t_lb
s
s
c
t_vt
c
p_bt
c
t_lt
c
s
s
c
s
c
p_sa
c
p_cp
s
c
t_vp
s
c
s
t_udb
p_rp
p_es
d
t_exit

1. If a client has entered the system, then eventually it will exit the system. This can be expressed as $\Phi_1 {=} G_s(\forall x)\big(p_{il}(x) \Rightarrow F_c\ p_{es}(x)\big)$.
2. If a client has entered the system, then it will not be in the place p_{il} after exiting the system. This can be expressed as $\Phi_2 {=} G_s(\forall x)\big(p_{es}(x) \Rightarrow \neg p_{il}(x)\big)$.
3. If a client has entered the system and it has chosen the mode of journey as flight, then eventually it will be in the place p_{cp}, where they share their payment information. This can be expressed as $\Phi_3 {=} G_s(\forall x)\big(p_{il}(x) \wedge p_{bf}(x) \Rightarrow F_c\ p_{cp}(x)\big)$.

4 If a client has entered the system and it has chosen the mode of journey as flight, and its journey is not yet validated by the server, then eventually it will be in the place p_{cp}, where they share their payment information to the server. This can be expressed as $\Phi_4 = G_s(\forall x)\big(p_{il}(x) \wedge p_{bf}(x) \wedge \neg p_{cp}(x) \Rightarrow F_c\ p_{cp}(x)\big)$.

5 If a client has entered the system, then eventually it will be in the place p_{cp}, after which it gets server approval. This can be expressed as $\Phi_5 = G_s(\forall x)\big(p_{il}(x) \Rightarrow F_c\ p_{cp}(x)\big)$.

6 If a client's payment has been validated, then it will not be in the place p_{cp}. This can be expressed as $\Phi_6 = G_s(\forall x)\big(p_{es}(x) \Rightarrow \neg p_{cp}(x)\big)$.

7 If a client is searching for a journey it can only choose either of the three flight, train or bus journeys. $\Phi_7 = G_s(\forall x)\big(p_{ss}(x) \Rightarrow F_c(p_{bf}(x) \vee p_{bt}(x) \vee p_{bb}(x))\big)$.

Experiments with UCSChecker

Table 1: Experiments on APS case study

Property ID	Maximum Nesting Depth of temporal operators	Number of clauses	time (s)
Ψ_1	2	3	0.32
Ψ_2	2	2	0.4
Ψ_3	2	2	0.3
Ψ_4	2	3	0.38

Experiments with UCSChecker

Table 2: Experiments on travel agency case study

Property ID	Maximum Nesting Depth of temporal operators	Number of clauses	time (s)
Φ_1	2	2	0.027
Φ_2	1	2	0.028
Φ_3	2	3	0.032
Φ_4	2	4	0.028
Φ_5	2	2	0.025
Φ_6	1	2	0.025
Φ_7	2	4	0.028

Bounded Semantics

$M, [x \mapsto a], i \models^k_x F_c \alpha$ iff
$\exists j : i \leq j \leq k,\ a \in V_j$ and $M, [x \mapsto a], j \models^k_x \alpha$.

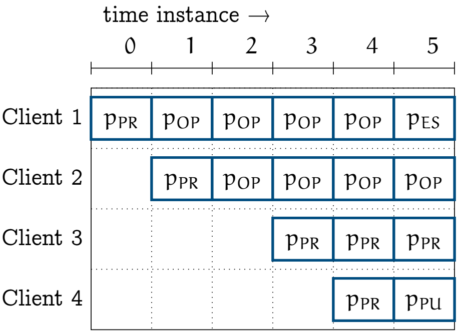


Figure: Snapshot with 4 clients

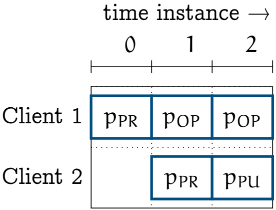


Figure: Snapshot with 2 clients

Given $\alpha = PR(x)$. The above formula is satisfiable for all clients in both figures.